QuizArena — Điểm cần hiểu cho vấn đáp (Phase 0 đến thứ Ba Phase 1)

Tổng hợp từ các buổi làm việc. Câu #12, #13, #14 có trong ngân hàng câu hỏi chính thức của đề; các câu còn lại là câu dự đoán sẽ bị hỏi. Đề yêu cầu bạn giải thích được từng dòng backend và hạ tầng, nên hãy đọc rồi tự trả lời bằng lời của mình, đừng học thuộc.

1. Kiến trúc và Docker

Vì sao tách `api`, `realtime`, `worker` dù chung codebase? (câu #13) Ba loại tải khác nhau. `api` xử lý request ngắn nên tải tỉ lệ với số request và CPU. `realtime` giữ kết nối WebSocket sống hàng giờ nên tải tỉ lệ với số kết nối đang mở. `worker` chạy việc dài hoặc hẹn giờ (gọi Gemini, đóng câu hỏi, ghi lô vào Postgres). Tách ra thì scale độc lập (dự kiến api 2–4, realtime 2–6, worker 1–3), deploy `api` không cắt WebSocket, job Gemini chậm không ảnh hưởng người đang chơi, và nginx cấu hình riêng cho WS. Cái giá: nhiều service hơn và phải có Redis Pub/Sub để các replica `realtime` nói chuyện với nhau. *Xem: docs/ARCHITECTURE.md mục 2.*

Vì sao đồng hồ trận năm ở `worker`, không ở `realtime`? (liên quan câu #3, #4) Nếu replica `realtime` đang giữ kết nối của host bị tắt giữa câu hỏi thì câu hỏi không được kẹt. Việc đóng câu là job hẹn giờ có `job_id` cố định, nên chạy đúng một lần (idempotent) và không phụ thuộc replica nào còn sống. *Xem: ARCHITECTURE mục 5.1; sẽ viết ADR-003.*

Vì sao chỉ nginx publish cổng? `docker compose ps` hiện `5432/tcp` nghĩa là gì? Chỉ nginx có `0.0.0.0:80->80`. Dòng như `5432/tcp` đứng một mình chỉ là cổng được khai báo (EXPOSE) trong image, không mở ra máy host. Postgres và Redis năm ở network `data` có `internal: true` nên không có đường ra Internet. `worker` đứng thêm ở network `app` (không internal) để sau này gọi được Gemini. Cần xem dữ liệu thì dùng `docker compose exec postgres psql ...`, không mở cổng.

Dockerfile: vì sao multi-stage, non-root, healthcheck bằng Python? Stage `builder` cài thư viện bằng `uv sync --frozen --no-dev --no-install-project`; chỉ copy `pyproject.toml` và `uv.lock` nên lớp này được cache, sửa code không phải cài lại. Stage `runtime` chỉ lấy `.venv` và mã nguồn nên nhỏ hơn và không có công cụ build. Chạy bằng user uid 10001 để nếu container bị chiếm thì kẻ tấn công không có quyền root. Image `slim`

không có `curl` nên HEALTHCHECK dùng `urllib` của Python; đường dẫn đổi được bằng biến `HEALTH_URL` (realtime dùng `/healthz`). `.dockerignore` loại `.env`, nếu thiếu thì secret nằm trong image.

`depends_on` hoạt động thế nào? Chuỗi `postgres` → `migrate` → `api` / `realtime` / `worker` → `nginx`. `service_healthy` chờ healthcheck của dịch vụ đạt; `service_completed_successfully` chờ service chạy một lần (`migrate`) thoát với mã 0. Nhờ vậy `api` chỉ khởi động khi schema đã sẵn sàng.

Vì sao service ứng dụng không có `container_name` và `ports`? Để `docker compose up --scale realtime=3` chạy được: tên hoặc cổng host cố định thì không nhân bản được. Bạn đã chứng minh bằng log nginx có hai `upstream_addr` khác nhau khi scale `api=2`.

Cấu hình Redis có gì đáng nói? Password đọc từ biến môi trường (`$$REDIS_PASSWORD` trong `sh -c`) để không lộ trong `docker inspect`. `maxmemory-policy noeviction`: Redis giữ trạng thái trận nên khi đầy bộ nhớ thì báo lỗi ghi chứ không âm thầm xóa dữ liệu. `appendonly yes` để giữ dữ liệu qua lần khởi động lại.

`docker-compose.override.yml` để làm gì, và cái bẫy image cũ? Nó tự được nạp khi chạy `docker compose up`, dùng cho dev (hot reload, polling cho ổ Windows). Production chạy `docker compose -f docker-compose.yml up -d` để bỏ qua nó. Override chỉ mount `./backend/app`, không mount `alembic/`, nên file migration mới chỉ vào container khi build lại image. Bạn đã gặp đúng lỗi này: `downgrade -1` báo "didn't produce 1 migrations" và `upgrade head` im lặng, vì container không thấy migration nào.

2. nginx

Làm sao scale replica mà không reload nginx? `resolver 127.0.0.11` (DNS nội bộ Docker) cộng `upstream { zone ...; server api:8000 resolve; }`: nginx hỏi lại DNS định kỳ (5 giây) và tự nhận IP replica mới. Cần nginx ≥ 1.27.3. Dòng `could not be resolved` thoáng qua lúc container được tạo lại là bình thường.

Vì sao `least_conn` cho `realtime`? Kết nối WebSocket sống rất lâu và nặng nhẹ khác nhau; round robin dễ dồn nhiều kết nối vào một replica. `least_conn` chọn replica đang có ít kết nối nhất.

Vì sao cần `proxy_read_timeout 3600s` ở `/ws/`? Mặc định là 60 giây: một kết nối WS im lặng quá 60 giây sẽ bị nginx cắt, nên WS rớt đều đặn. Đi kèm là header `Upgrade` / `Connection` (qua `map`) và `proxy_buffering off`.

Vì sao `limit_conn` là 50 mà không phải 20? (câu #12) Một lớp học dùng chung Wi-Fi ra Internet bằng một địa chỉ IP (NAT), nên nginx thấy cả lớp là một IP; 20 sẽ chặn oan lớp 40 người. 50 (và `burst=60` ở `auth/*`, `games/join`) là con số khởi điểm; bạn phải đo bằng load test ở Phase 5 và tự lý giải trong báo cáo. Trên Docker Desktop mọi request local đều thấy IP `172.20.0.1`, nên thử nghiệm local coi cả máy là một người.

`X-Forwarded-For` đặt thế nào, vì sao? `proxy_set_header X-Forwarded-For $remote_addr;`, không dùng `$proxy_add_x_forwarded_for` (nối thêm vào giá trị client tự gửi). Nginx là lớp ngoài cùng nên tin `$remote_addr`; như vậy client không giả IP để né rate limit.

Vì sao có hai `location` cho API, và regex hay prefix thắng? Location kiểu regex (`~`) thắng prefix (`/api/`). Vì vậy `auth/*` và `games/join` rơi vào zone `api_sensitive` (5 request/giây, chống dò mật khẩu và dò PIN 6 số), phần còn lại vào `api_general` (30 request/giây). Hai khối phải lặp các `proxy_set_header` vì nginx không thừa kế chúng giữa các location cùng cấp. Route upload tài liệu AI là location lồng nhau (nhận tới 10 MB): nó thừa kế `proxy_set_header` nhưng phải tự khai báo `proxy_pass`.

Log nginx và request_id? Log JSON có `request_id`, `upstream_addr`, `upstream_response_time`. Dùng `$uri` thay vì `$request_uri` để query string (có thể chứa `?ticket=` hay PIN) không vào log. nginx sinh `$request_id` và gửi xuống app qua `X-Request-ID`; app dùng lại giá trị đó nên một request có cùng ID ở cả hai log. `proxy_hide_header X-Request-ID` tránh trả hai header trùng ra client.

Vì sao chặn `/metrics`, `/docs`, `/internal` bằng 404? Chúng không được proxy, nhưng nginx có SPA fallback (`try_files ... /index.html`) nên nếu không chặn thì `/metrics` trả 200 với trang HTML. Chặn rõ ràng là phòng thủ nhiều lớp (Prometheus scrape qua mạng nội bộ, không qua nginx).

3. Nền tảng backend

Cấu hình bằng pydantic-settings: vì sao có các tùy chọn đó? `SecretStr` để secret in ra log thành `**********`. `extra="ignore"` vì `.env` có `POSTGRES_*`, `REDIS_PASSWORD` mà chỉ compose dùng. `env_ignore_empty=True` để `GEMINI_RPM=` (chuỗi rỗng) coi như chưa đặt thay vì lỗi ép kiểu. `hide_input_in_errors=True` để lỗi validate không in giá trị secret vào log. `jwt_secret` bắt buộc ≥ 32 ký tự, và ở `APP_ENV=prod` giá trị còn `change-me` bị từ chối. Mục tiêu: lỗi cấu hình làm app dừng ngay lúc khởi động, không chạy tiếp rồi lỗi giữa chừng.

Vì sao `JWT_SECRET` phải ngẫu nhiên, và vì sao mỗi secret một giá trị? Server dùng nó để ký token; ai biết hoặc đoán được nó thì tự tạo được token hợp lệ, kể cả giả `ADMIN`. Chuỗi như `abc123` bị dò từ điển trong vài giây; dùng `openssl rand -hex 32`. Postgres, Redis, JWT mỗi thứ một giá trị riêng để lộ một cái không kéo theo cái khác. Đổi `JWT_SECRET` thì mọi token đang lưu hành mất hiệu lực.

Middleware: vì sao viết ASGI thuần, và thứ tự thế nào? Không dùng `BaseHTTPMiddleware` vì nó chạy request trong task khác (làm hỏng `contextvar`) và không hợp WebSocket. `RequestIDMiddleware` năm ngoài cùng để mọi log, kể cả access log, đã có `request_id`. Thứ tự đích (ngoài → trong): Sentry → RequestID → Log → CORS → Metrics → Exception; hiện mới làm RequestID và Log. Request-ID nhận header `X-Request-ID` nếu khớp `^[A-Za-z0-9._-]{8,64}$` (tránh bị chèn ký tự lạ vào log), không thì sinh `uuid4().hex`. Không reset `contextvar` ở cuối vì handler lỗi 500 của Starlette chạy ngoài middleware và vẫn cần giá trị; an toàn vì uvicorn xử lý mỗi request trong một task riêng.

Chuẩn lỗi thống nhất là gì? Mọi lỗi có dạng `{"error": {"code", "message", "request_id"}}`: 404, 405, 422 (kèm `details`), lỗi nghiệp vụ `AppError`, và 500. Lỗi 500 chỉ ghi traceback vào log (có `request_id`) và không trả chi tiết nội bộ cho client.

`/healthz` khác `/readyz` thế nào? `healthz` (liveness) chỉ cho biết tiến trình còn chạy, không kiểm tra phụ thuộc. `readyz` (readiness) kiểm tra thật Postgres và Redis với timeout 2 giây, trả 503 nếu một trong hai lỗi: ngừng nhận việc nhưng không cần giết tiến trình.

Phân tầng code và các mẫu thiết kế (câu #14)? `api / realtime / workers` → `services` → `repositories` → `models`; kiểm tra quyền sở hữu ở tầng `services`. Mẫu dự kiến: State (trạng thái trận), Strategy (cách tính điểm), Adapter (Gemini gateway, có bản giả để test), Observer (Redis Pub/Sub), Command (thông điệp WS). Hiện chưa mẫu nào được cài; chúng xuất hiện ở Phase 2–3, nên đừng nói đã dùng khi chưa viết.

Alembic async hoạt động ra sao? `env.py` lấy URL từ `get_settings()` (không để trong `alembic.ini`) và dùng `create_async_engine` với `NullPool` (migration là tiến trình chạy một lần, không cần pool). Migration đầu bật extension `citext` trước khi tạo bảng vì Alembic không tự tạo extension; `downgrade` chỉ xóa bảng, giữ lại extension. Quy ước đặt tên ràng buộc (`NAMING_CONVENTION`) giúp tên ổn định để `downgrade` xóa đúng. `alembic check` xác nhận model và migration khớp nhau. Kiểm tra bắt buộc: `upgrade head` → `downgrade -1` → `upgrade head`.

Vì sao email dùng `citext`? So sánh không phân biệt hoa thường ngay ở DB, nên `UNIQUE` chặn cả `a@x.com` lẫn `A@X.com` (FR-AUTH-01), kể cả khi hai request đến cùng lúc. App còn chuẩn hóa email về chữ thường.

CI chạy những gì, vì sao như vậy? ruff (lint và format) → mypy → migration up/down/up → pytest với Postgres và Redis thật (service container) → `docker build`; gitleaks chạy song song. `REQUIRE_SERVICES=1` biến test cần DB thành LỖI nếu thiếu dịch vụ (máy local thì bỏ qua). Phiên bản Action đã được kiểm tra thật trên GitHub (`checkout@v7`, `setup-uv@v10.3.2`, `build-push-action@v7`). `concurrency` hủy lần chạy cũ khi có commit mới.

pre-commit hoạt động ra sao, và những lỗi đã gặp? Hook chạy trên các file đã stage: ruff, hook chuẩn (khoảng trắng, cuối file, private key) và gitleaks. Khi hook tự sửa file thì commit bị dừng để bạn xem lại; nếu còn file nửa stage, pre-commit stash rồi hoàn tác ("Rolling back fixes"), cách xử lý là stage hết rồi chạy `pre-commit run --all-files` hai lần. Lỗi `WinError 3` khi dựng môi trường hook là do Python bản Microsoft Store ảo hóa AppData khiến đường dẫn cache của `virtualenv` vượt 260 ký tự; `pre-commit-uv` dựng môi trường bằng `uv` nên tránh được.

Quản lý secret thế nào? `.env` nằm trong `.gitignore` (kiểm tra bằng `git check-ignore -v .env`), `.env.example` chỉ có giá trị mẫu. Commit secret là điểm liệt; nếu lỡ lộ thì xóa file là chưa đủ vì nội dung còn trong lịch sử git, phải thu hồi và tạo key mới. Gitleaks bắt secret ở pre-commit và CI; chạy `gitleaks detect` một lần trước nghiệm thu (mục §9.2 của đề).

4. Xác thực (Phase 1)

Vì sao argon2id, và các chi tiết đi kèm? Hàm băm mật khẩu phải chậm và tốn bộ nhớ để dò thử đắt; argon2id có salt riêng mỗi lần băm (hai lần băm cùng mật khẩu cho kết quả khác nhau). Băm nặng CPU nên chạy trong `asyncio.to_thread` để không chặn event loop, và chạy NGOÀI transaction DB để không giữ kết nối trong lúc tính. Mật khẩu giới hạn 8–128 ký tự (có cả chữ và số): giới hạn trên chặn kiểu gửi mật khẩu khổng lồ để làm argon2 tốn tài nguyên.

Làm sao đăng nhập không lộ email nào đã đăng ký? Email không tồn tại và sai mật khẩu trả cùng một mã lỗi `INVALID_CREDENTIALS` và cùng thông điệp. Thời gian cũng như nhau vì khi không có user, code vẫn băm với một "hash giả". "Tài khoản bị khóa" (403) chỉ được nói sau khi đã đúng mật khẩu.

Cấu trúc JWT, và vì sao có `type`, `ver`, `jti`? Access token (HS256, sống 15 phút) chứa `sub`, `roles`, `permissions`, `iat`, `exp`, `jti`, `ver`, `type="access"`. `type` ngăn guest token dùng như access token (và ngược lại). `jti` là mã duy nhất của token để đưa vào denylist khi logout (thứ Tư). `ver` là `token_version` của user, tăng khi đổi mật khẩu hoặc đổi vai trò để vô hiệu token cũ. Khi giải mã luôn chỉ định `algorithms=["HS256"]` (chống đổi thuật toán và `alg=none`); đã có test cho `alg=none`, sửa payload thành `ADMIN`, chữ ký sai, thiếu trường.

Ba loại token khác nhau thế nào? Vì sao guest không có refresh? Access (ngắn, trong body), refresh (dài 7 ngày theo AU-2, trong cookie, server lưu bản băm), guest (không có tài khoản: chỉ quyền `game:play`, gắn một `game_id`, tối đa 3 giờ). Khách không có tài khoản để thu hồi và chỉ chơi một trận, nên token ngắn hạn là đủ và không cần refresh. (Câu 2 của ADR-001.)

Vì sao refresh token chỉ lưu SHA-256, không dùng argon2? DB không bao giờ chứa token gốc, nên lộ DB cũng không dùng được token. SHA-256 là đủ vì token là chuỗi ngẫu nhiên entropy cao (không như mật khẩu người dùng đặt), argon2 chậm sẽ vô ích. Tra cứu bằng cách băm token nhận được rồi so với cột `token_hash` (UNIQUE).

Giải thích rotation và reuse detection (AU-4, AU-5). Mỗi lần refresh, token cũ bị đánh dấu `revoked_at` và token mới được cấp trong cùng `family_id` (mọi token từ một lần đăng nhập). Nếu một token ĐÃ bị xoay mà vẫn được gửi lại, nghĩa là có hai bên cùng giữ nó (nghi bị đánh cắp), nên thu hồi cả family và trả 401. Ví dụ: R1 → R2 hợp lệ; kẻ cắp gửi R1 → 401 và R2 cũng bị thu hồi; người dùng phải đăng nhập lại. Phiên đăng nhập ở thiết bị khác (family khác) không bị ảnh hưởng.

Hai request refresh cùng lúc thì sao? (chống đua) Dùng một câu `UPDATE ... WHERE revoked_at IS NULL AND expires_at > now() RETURNING ...`: DB chỉ trả dòng cho đúng một request; request kia không nhận được gì nên không bao giờ sinh ra hai token con. Nếu tách thành "SELECT rồi UPDATE" thì cả hai cùng thắng.

Vì sao phải ném lỗi sau khi commit? Nếu `raise` bên trong `async with session.begin()` thì transaction rollback, việc thu hồi family biến mất trong khi request vẫn trả 401. Code lưu lỗi vào biến, thoát khối `with` (commit) rồi mới ném. Có test kiểm tra DB thật sự không còn token nào hiệu lực.

Cookie refresh có 4 thuộc tính nào, mỗi cái chống gì? `HttpOnly`: JavaScript không đọc được nên XSS không lấy được token. `Secure`: chỉ gửi qua HTTPS (Chrome và Firefox thường vẫn nhận trên localhost, Safari thì không). `SameSite=Strict`: không gửi kèm request từ trang khác, chống CSRF. `Path=/api/v1/auth`: chỉ gửi tới nhóm endpoint auth, giảm bề mặt lộ. Khi refresh thất bại, server xóa cookie.

Đăng ký: vì sao bắt lỗi ở DB thay vì kiểm tra email trước? "SELECT rồi INSERT" có khe hở: hai request cùng lúc đều thấy "chưa có". Để ràng buộc `UNIQUE` quyết định rồi bắt `IntegrityError` thành 409; test chạy 6 request đồng thời cho đúng 1 thành công và 5 lỗi 409. Người dùng mới có vai trò `PLAYER` (cần `seed` đã chạy, nếu không báo `ROLES_NOT_SEEDED`).

Hai tab cùng refresh một lúc có vấn đề gì? Request thua cuộc bị coi là dùng lại token và có thể làm mất phiên. Đây là cái giá của reuse detection nghiêm ngặt, nên frontend phải làm single-flight: chỉ một request refresh tại một thời điểm, các request khác chờ kết quả đó (`ARCHITECTURE.md` mục 5.3).

RBAC: vì sao ma trận quyền nằm trong DB mà code vẫn có `ROLE_PERMISSIONS`? Lúc chạy, hệ thống đọc quyền từ DB và từ claim `permissions` trong JWT, không hard-code vai trò trong logic phân quyền (FR-AUTHZ-01). `ROLE_PERMISSIONS` chỉ là dữ liệu mẫu cho `seed` và nguồn đối chiếu cho test. `seed` dùng `INSERT ... ON CONFLICT DO NOTHING` nên chạy nhiều lần vẫn an toàn (lần đầu 4 vai trò, 10 quyền, 21 lượt gán; lần hai là 0). Ma trận: GUEST 1 quyền, PLAYER 3, HOST 7, ADMIN 10.

ADR-001: ba câu hỏi và hướng trả lời (1) Quyền năm trong JWT thì thu hồi quyền có hiệu lực sau bao lâu? Tối đa tới khi access token hết hạn (15 phút), trừ khi kiểm tra `ver` mỗi request; chọn một cách và nói rõ cái giá. (2) Vì sao guest không có refresh? Xem trên. (3) Guest token bị lộ thì thiệt hại tối đa? Kẻ cầm token chỉ đóng vai khách đó trong đúng một trận, chỉ có `game:play`, tối đa 3 giờ, không gọi được API khác. Bạn tự viết bằng lời của mình.

5. Kiểm thử và quy trình

Vì sao test dùng Postgres và Redis thật? Nhiều lỗi chỉ lộ khi có DB thật: tranh chấp đồng thời, ràng buộc UNIQUE, thu hồi family trong transaction, `citext`. Test chạy với `httpx.ASGITransport` kèm lifespan của app và dọn dữ liệu thử sau mỗi test. Hiện có 76 test, chạy 3 lần liên tiếp đều qua.

Test nào thể hiện các điểm khó nhất?
`test_hai_refresh_dong_thoi_chi_mot_thanh_cong` (chống đua),
`test_dung_lai_refresh_token_cu_thu_hoi_ca_family` (commit trước khi ném),
`test_dang_ky_dong_thoi_cung_email_chi_mot_thanh_cong`,
`test_refresh_token_chi_luu_ban_bam_sha256`,
`test_cookie_refresh_co_dung_4_thuoc_tinh`.

6. Con số cần nhớ

Mục	Giá trị
Access token	15 phút
Refresh token	7 ngày (đề AU-2; máy bạn từng là 30 do `.env`)
Guest token	tối đa 3 giờ
WS ticket	30 giây, dùng một lần (làm thứ Năm)
Mật khẩu	8–128 ký tự, có chữ và số
`limit_conn` WS	50 mỗi IP (đề ví dụ 20, cần lý giải)
`limit_req`	30 request/giây chung; 5 request/giây cho auth và join
Upload tài liệu AI	tối đa 10 MB
Vai trò / quyền / lượt gán	4 / 10 / 21
Số replica dự kiến	api 2–4, realtime 2–6, worker 1–3

7. Chưa làm (đừng nói là đã có)

Logout, denylist `jti`, kiểm tra `ver`, khóa tạm khi sai mật khẩu nhiều lần, `logout-all` (thứ Tư). RBAC dependency, guest token và `games/join`, ws-ticket, CLI `create-admin` (thứ Năm). ADR-001 và SRS v0.2 (thứ Sáu). CORS, Metrics, Sentry, TLS, tinh chỉnh rate limit bằng load test, giao thức WebSocket thật (Phase 3), giám sát.

8. Những lỗi đã gặp (kể ra là điểm cộng)

Chạy lệnh bash trong PowerShell; Python Microsoft Store làm pre-commit lỗi `WinError 3`; image Docker cũ khiến Alembic không thấy migration; `REFRESH_TOKEN_DAYS=30` trong `.env` làm test cố định 7 ngày fail; `env.py` "bản lai" giữa template và code mới; formatter VS Code (autopep8) đánh nhau với `ruff format`; tên `dockerfile` viết thường lệch trên Linux; file nửa stage làm pre-commit hoàn tác.

9. Tự kiểm tra (trả lời thành tiếng, mỗi câu một phút)

1. Vẽ lại sơ đồ 7 container và nói đường đi của một request WebSocket.
2. Vì sao scale `realtime` từ 2 lên 3 không cần reload nginx?
3. Tại sao `limit_conn 20` có thể chặn oan một lớp học?
4. Điều gì xảy ra nếu bạn quên `proxy_read_timeout`?
5. Kể tên 3 thứ pydantic-settings giúp bạn bắt lỗi sớm.
6. Vì sao request ID phải xuyên suốt nginx → app → log?
7. Vì sao `login` không để lộ email nào tồn tại?
8. Giải thích rotation và reuse detection bằng ví dụ R1, R2.
9. Hai refresh đồng thời được xử lý ra sao ở mức câu SQL?
10. Vì sao phải ném lỗi sau khi commit?
11. Mỗi thuộc tính của cookie refresh chống lại tấn công nào?
12. `ROLE_PERMISSIONS` trong code có mâu thuẫn với "không hard-code role" không?